DigitalOcean Deployment Guide for LoanInNeed

This document outlines the deployment strategy for the LoanInNeed application on a DigitalOcean Droplet.

1. System Architecture

The application consists of three main components:

- **Frontend**: Next.js application (`lin-frontend`).
- **Backend**: Node.js/Express application (`Backend`).
- **Database**: PostgreSQL (Running in a Docker container).

2. Prerequisites

- **Server**: DigitalOcean Droplet (Recommended: Ubuntu 22.04 LTS, at least 2GB RAM for build processes).
- **Domain**: A domain name pointed to the Droplet's IP address.
- **Docker & Docker Compose**: For containerized deployment.

3. Environment Variables

Backend (`Backend/.env`)

Create this file in the `Backend` directory.

```
NODE_ENV=production
PORT=5000
DATABASE_URL=postgresql://loaninneed:<password>@postgres:5432/loaninneed_db?
schema=public
# Security
JWT_SECRET=your_super_secure_jwt_secret
# Storage (S3 - Optional if using Local)
AWS_ACCESS_KEY_ID=your_aws_key
AWS_SECRET_ACCESS_KEY=your_aws_secret
AWS_REGION=us-east-1
AWS_S3_BUCKET_NAME=your_bucket_name
# Third Party
TWILIO_ACCOUNT_SID=your_twilio_sid
TWILIO_AUTH_TOKEN=your_twilio_token
TWILIO_PHONE_NUMBER=your_twilio_number
SUPABASE_URL=your_supabase_url
SUPABASE_SERVICE_KEY=your_supabase_key
```

Frontend (`lin-frontend/.env.local`)

Create this file in the `lin-frontend` directory.

```
NEXT_PUBLIC_API_URL=https://api.yourdomain.com
```

4. Deployment Steps (Docker Compose Method)

Step 1: Prepare the Server

SSH into your DigitalOcean Droplet and install Docker:

```
sudo apt update
sudo apt install docker.io docker-compose -y
```

Step 2: Clone Repository

```
git clone <your-repo-url>
cd LoanInNeed
```

Step 3: Create Frontend Dockerfile

Create a file named `Dockerfile` inside the `lin-frontend` directory:

```
# lin-frontend/Dockerfile
FROM node:18-alpine AS base

# Install dependencies only when needed
FROM base AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

# Rebuild the source code only when needed
FROM base AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

# Production image, copy all the files and run next
FROM base AS runner
WORKDIR /app
ENV NODE_ENV production
RUN addgroup --system --gid 1001 nodejs
RUN adduser --system --uid 1001 nextjs

COPY --from=builder /app/public ./public
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./.next/static

USER nextjs
```

```
EXPOSE 3000
ENV PORT 3000
CMD ["node", "server.js"]
```

Note: You need to update `next.config.ts` (or `.js`) to include `output: 'standalone'` for this Dockerfile to work efficiently.

Step 4: Create Production Docker Compose

Create a `docker-compose.prod.yml` in the root (or `Backend`) directory to orchestrate all services.

```
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    container_name: lin-postgres
    restart: always
    environment:
      POSTGRES_USER: loaninneed
      POSTGRES_PASSWORD: secure_password_here
      POSTGRES_DB: loaninneed_db
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - lin-network

  backend:
    build:
      context: ./Backend
      dockerfile: Dockerfile
      target: production
    container_name: lin-backend
    restart: always
    ports:
      - "5000:5000"
    environment:
      -
DATABASE_URL=postgresql://loaninneed:secure_password_here@postgres:5432/loaninneed
_db?schema=public
      - NODE_ENV=production
      # Add other env vars here or use env_file
    env_file:
      - ./Backend/.env
    depends_on:
      - postgres
    networks:
      - lin-network

  frontend:
    build: ./lin-frontend
```

```
    container_name: lin-frontend
    restart: always
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
    networks:
      - lin-network

volumes:
  postgres_data:

networks:
  lin-network:
    driver: bridge
```

Step 5: Start Services

```
docker-compose -f docker-compose.prod.yml up -d --build
```

This will build the images and start the containers.

5. Reverse Proxy (Nginx) & SSL

It is highly recommended to use Nginx as a reverse proxy to handle SSL and route traffic.

Install Nginx

```
sudo apt install nginx -y
```

Configure Nginx

Create a config file `/etc/nginx/sites-available/loaninneed`:

```
server {
    server_name yourdomain.com; # Frontend Domain

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

```
server {
    server_name api.yourdomain.com; # Backend Domain

    location / {
        proxy_pass http://localhost:5000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

Enable the site and restart Nginx:

```
sudo ln -s /etc/nginx/sites-available/loaninneed /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx
```

Setup SSL (Certbot)

```
sudo apt install certbot python3-certbot-nginx -y
sudo certbot --nginx -d yourdomain.com -d api.yourdomain.com
```

6. Maintenance & Monitoring

- **Logs**:
  - Backend: `docker logs -f lin-backend`
  - Frontend: `docker logs -f lin-frontend`
- **Database Backups**:
  - Run inside container: `docker exec -t lin-postgres pg_dumpall -c -U loaninneed > dump_$(date +%Y-%m-%d).sql`